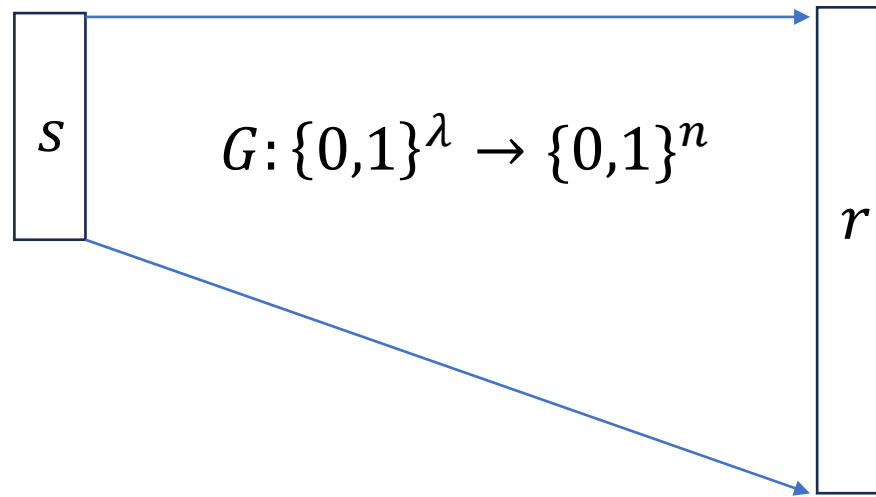


COS 433: Cryptography

Reminder:

- Problem Set 1 due 9/23 on Gradescope.

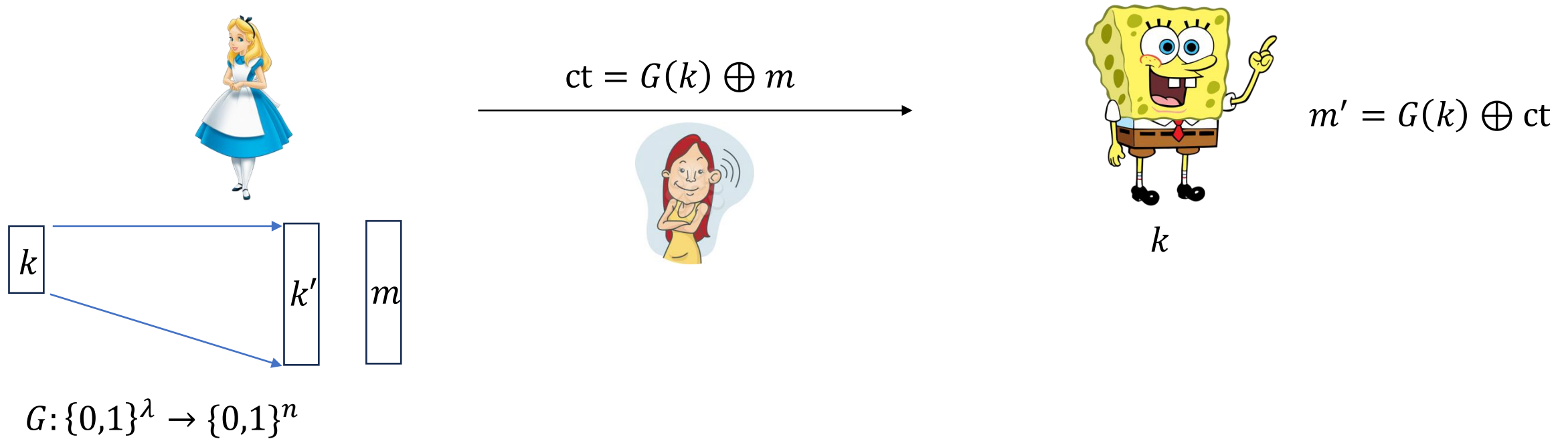
Pseudorandom Generators



Definition: An **efficiently computable** function family $G = \{G^{(\lambda)}: \{0,1\}^\lambda \rightarrow \{0,1\}^{n(\lambda)}\}_\lambda$ is a **pseudorandom generator (PRG)** if it satisfies two additional properties:

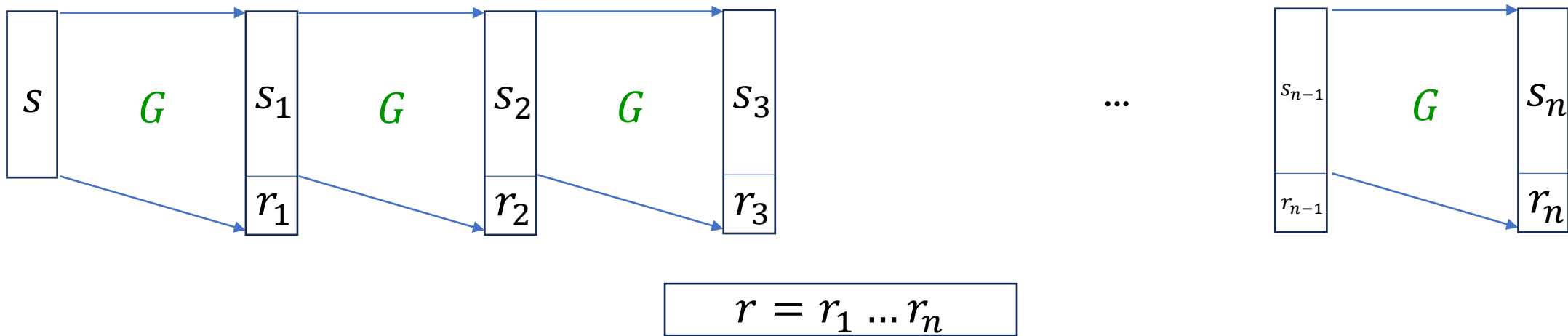
- Stretch: $n(\lambda) > \lambda$ for all λ
- Pseudorandomness: $G(U_\lambda) \approx_c U_n$ (uniform n bit string)

Encryption from PRGs



Theorem: if G is a PRG, then this is a secure one-time encryption scheme!

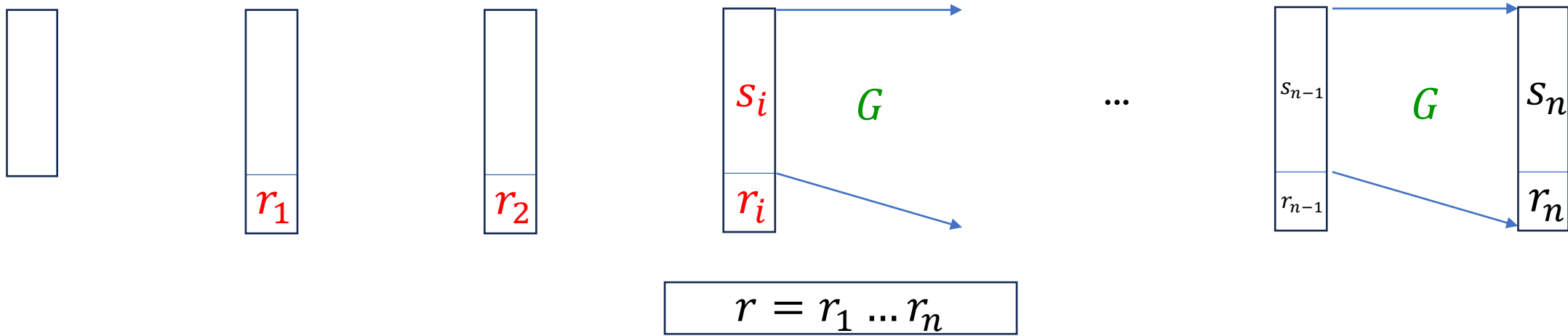
PRG Length Extension



Theorem: 1 bit of “stretch” is enough!

If $G: \{0,1\}^\lambda \rightarrow \{0,1\}^{\lambda+1}$ is a PRG, then for any $n(\lambda) = \lambda^c$, there exists $G': \{0,1\}^\lambda \rightarrow \{0,1\}^{n(\lambda)}$

PRG Length Extension

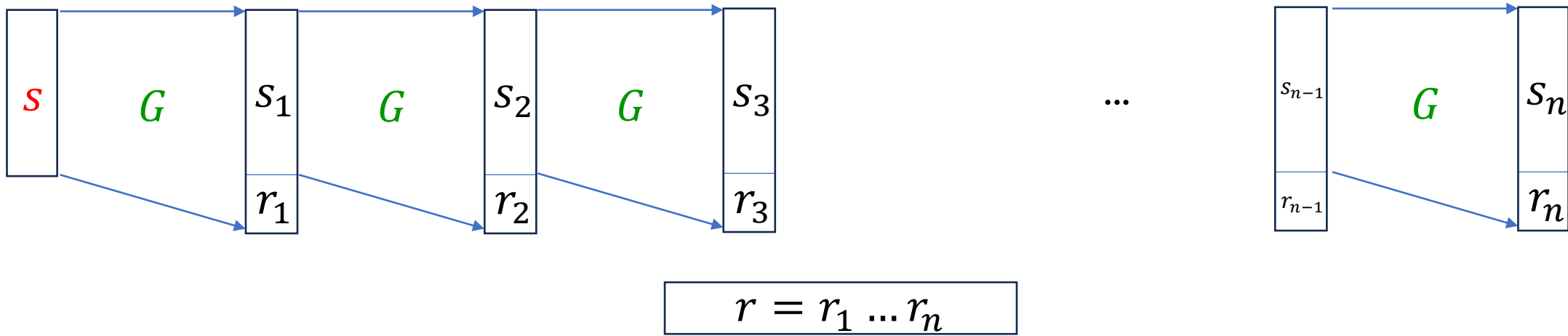


Claim: $G'(s) \approx_c r$

Proof: Define the following distributions

$$X_i^{(\lambda)} = U_i \times G'_i(U_\lambda)$$

PRG Length Extension

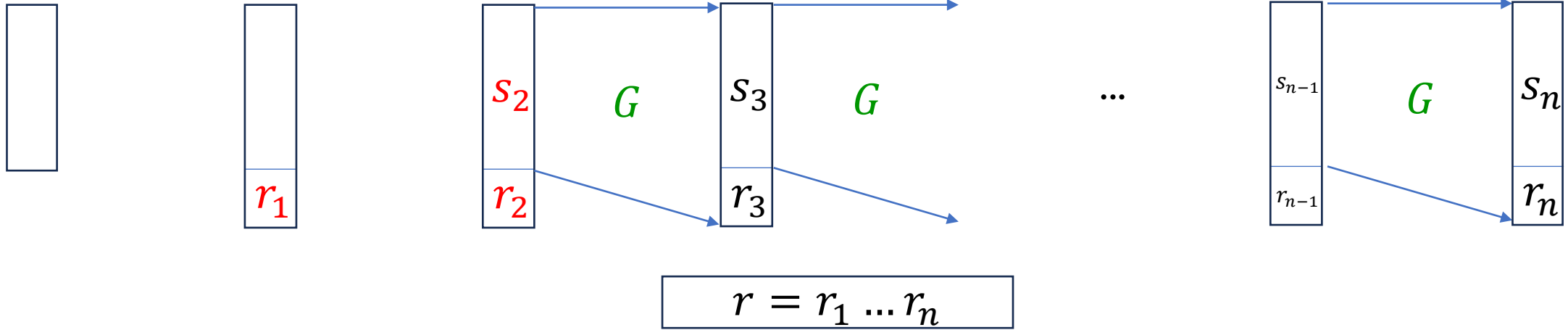


Claim: $G'(s) \approx_c r$

Proof: Define the distributions $X_i^{(\lambda)} = U_i \times G'_i(U_\lambda)$. Then, $\exists i$ such that

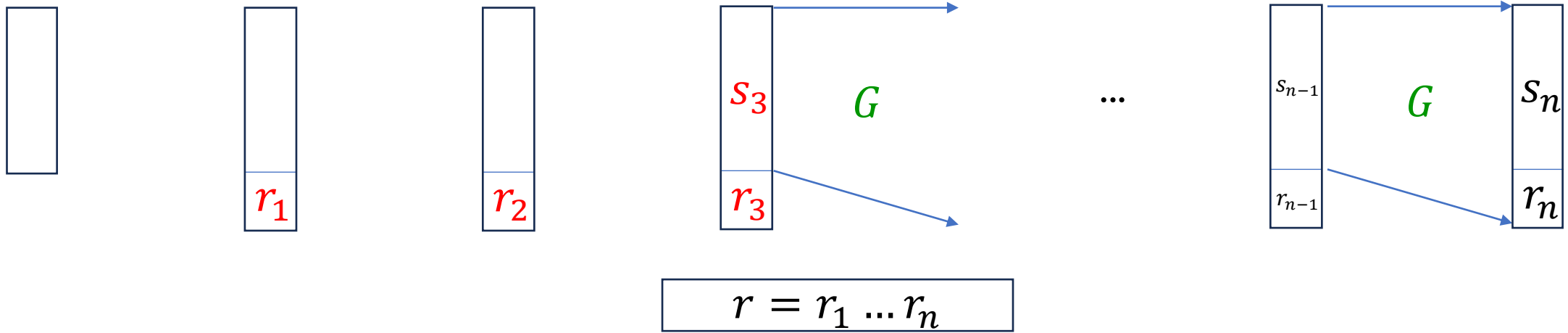
$$\left| \mathbf{E} \left[D \left(X_i^{(\lambda)} \right) \right] - \mathbf{E} \left[D \left(X_{i-1}^{(\lambda)} \right) \right] \right| \geq \frac{1}{n(\lambda)} \cdot \left| \mathbf{E}[D(G'(s))] - \mathbf{E}[D(r)] \right|$$

PRG Length Extension



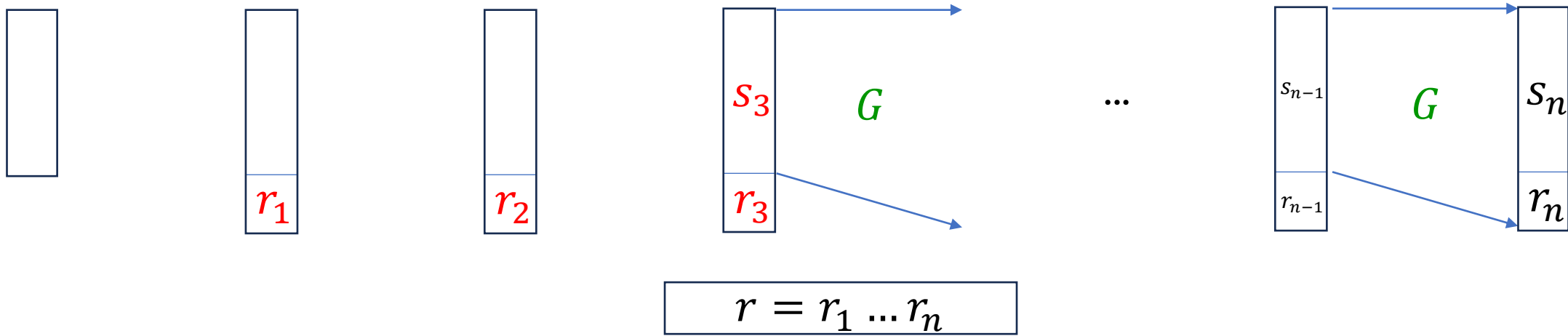
$$X_{i-1}^{(\lambda)} = U_{i-1} \times G'_{i-1}(U_\lambda)$$

PRG Length Extension



$$X_i^{(\lambda)} = U_i \times G'_i(U_\lambda)$$

PRG Length Extension



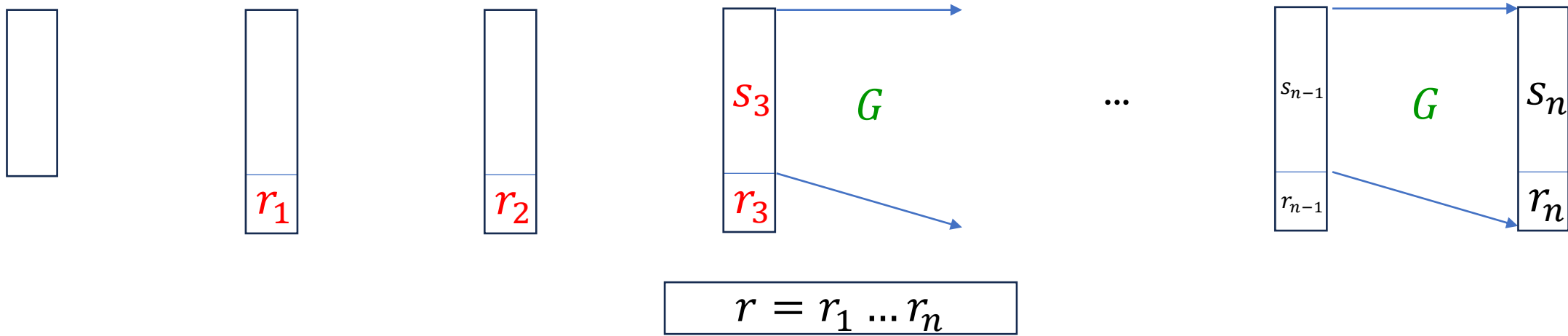
Proof: If G' is insecure, we will show that G is broken.

Define:

$$D_i^{(\lambda)}(x) = D(r_1, \dots, r_{i-1}, x_{\lambda+1}, G'_i(x_{1:\lambda}))$$

$$\left| \mathbf{E} \left[D_i^{(\lambda)}(r_i, s_i) \right] - \mathbf{E} \left[D_i^{(\lambda)}(G(s_{i-1})) \right] \right| = \left| \mathbf{E} \left[D \left(X_i^{(\lambda)} \right) \right] - \mathbf{E} \left[D \left(X_{i-1}^{(\lambda)} \right) \right] \right|$$

PRG Length Extension



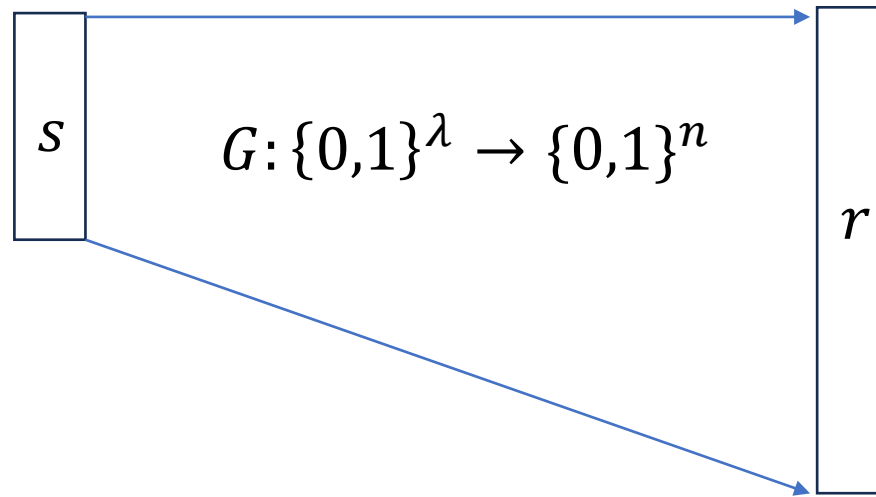
Proof: If G' is insecure, we will show that G is broken.

Define:

$$D_i^{(\lambda)}(x) = D(r_1, \dots, r_{i-1}, x_{\lambda+1}, G'_i(x_{1:\lambda}))$$

$$\left| \mathbf{E} \left[D_i^{(\lambda)}(r_i, s_i) \right] - \mathbf{E} \left[D_i^{(\lambda)}(G(s_{i-1})) \right] \right| \geq \frac{1}{n(\lambda)} \cdot |\mathbf{E}[D(G'(s))] - \mathbf{E}[D(r)]|$$

Pseudorandom Generators

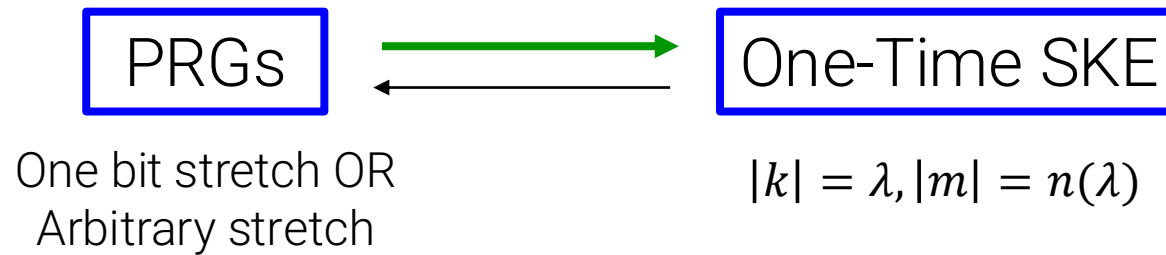


Theorem: 1 bit of “stretch” is enough!

If $G: \{0,1\}^\lambda \rightarrow \{0,1\}^{\lambda+1}$ is a PRG, then for any $n(\lambda) = \lambda^c$, there exists $G': \{0,1\}^\lambda \rightarrow \{0,1\}^{n(\lambda)}$

Relationships between Cryptographic Primitives

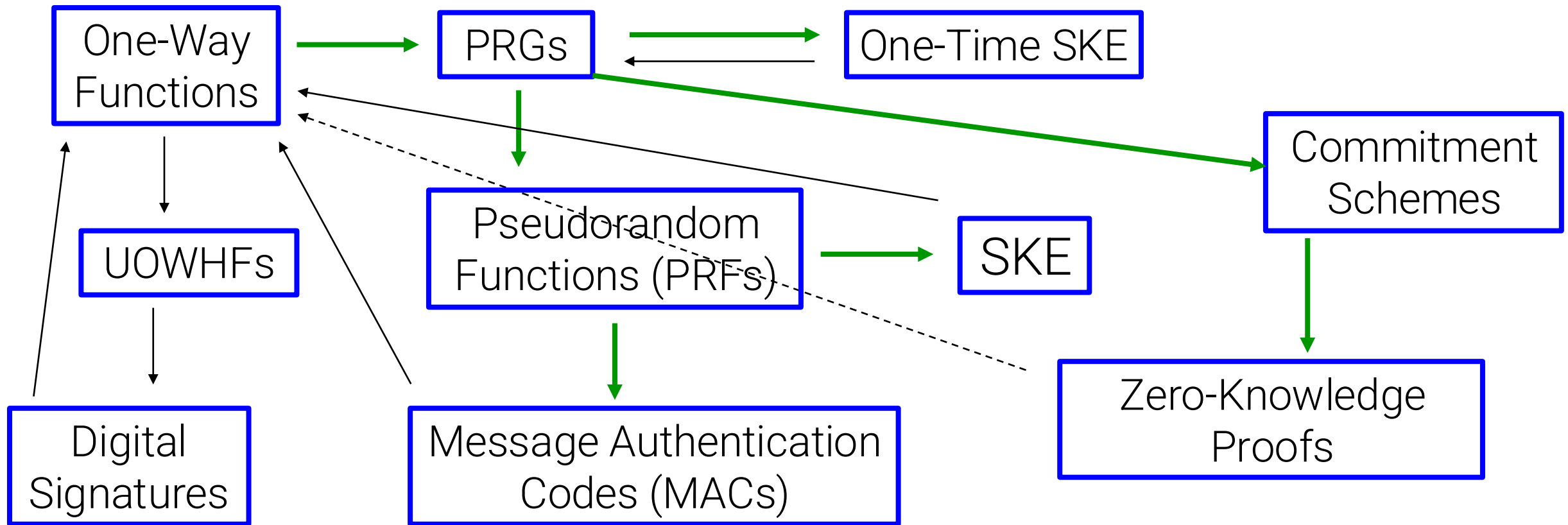
Theorem: the following are **existentially equivalent** (we won't prove this)



To build one-time SKE (for long messages), it is **necessary and sufficient** to construct a PRG!

Relationships between Cryptographic Primitives

Theorem: the following are **existentially equivalent** (we won't prove this in full)



“The PRG Conjecture”

Main Conjecture: PRGs with one bit of stretch exist

If true, we have secret-key cryptography!

“The PRG Conjecture”

Main Conjecture: PRGs with one bit of stretch exist

Evidence for conjecture:

- 1) **Equivalent** to many other conjectures (including weaker looking ones)
- 2) “**Generic attack**” that treats G as a black box shouldn’t be expected to work (HW1)
- 3) **Concrete candidates!** Have withstood attacks for many decades (or centuries...)
 - from practice (AES)
 - from theory (next slide)

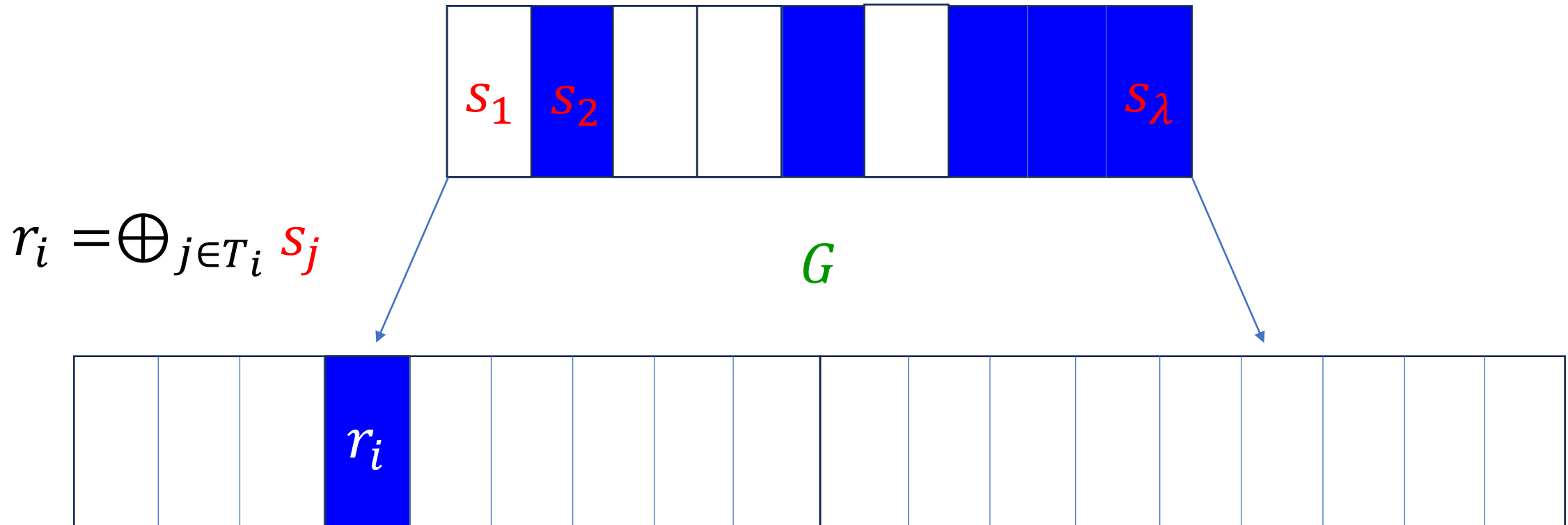
Let's build a PRG!

The Subset Sum Generator

s_1	s_2							s_λ
-------	-------	--	--	--	--	--	--	-------------

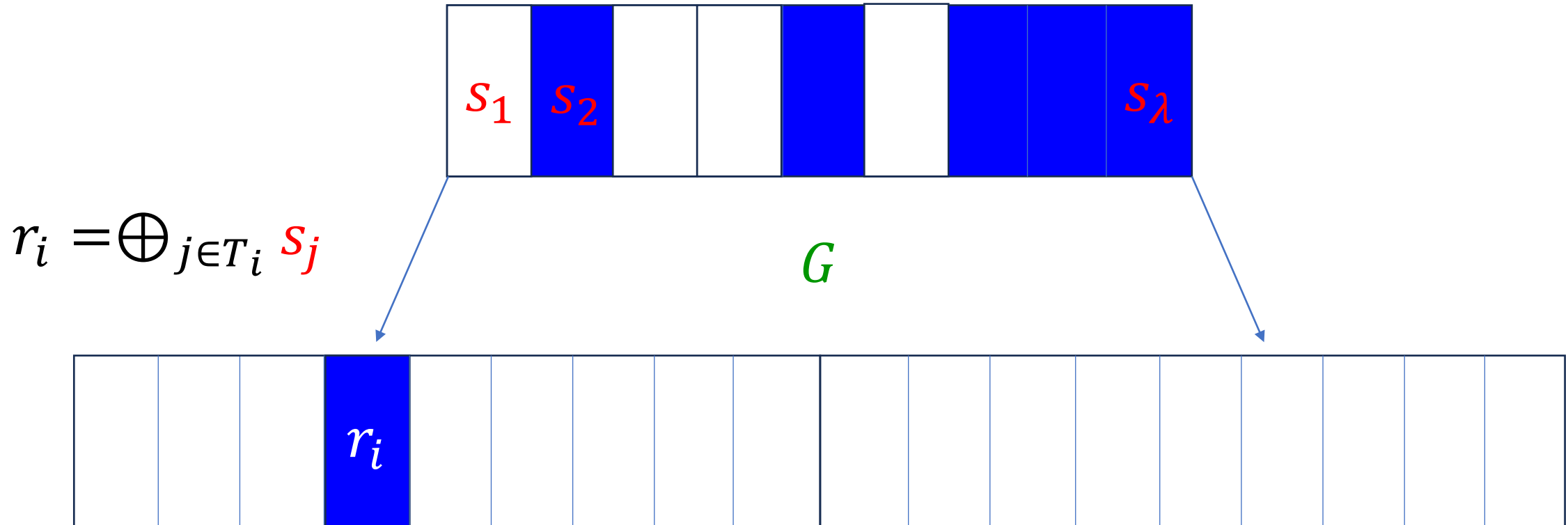
The Subset Sum Generator

A subset sum generator is described by a collection of sets $T_1, \dots, \mathbf{T_i}, \dots, T_{n(\lambda)} \subset [\lambda]$



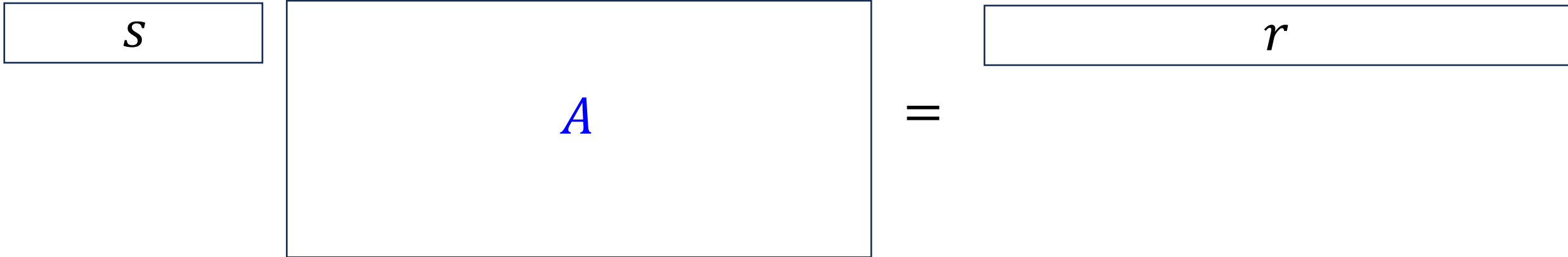
The Subset Sum Generator

Very commonly used in practice... but not for cryptography!



The Subset Sum Generator

$$G_A(\boxed{s}) = sA \pmod{2}$$



The Subset Sum Generator

$$G_A(\boxed{s}) = sA \pmod{2}$$

Poly-time attack:

- Given $y \in \{0,1\}^n$, check if $sA \equiv y \pmod{2}$ has a solution (“column reduction”)
- In the pseudorandom case, there will always be a solution!
- But for a uniformly random r , $sA \equiv r \pmod{2}$ is solvable w.p. at most $2^{\lambda-n}$

Takeaway: need some non-linearity in your PRG

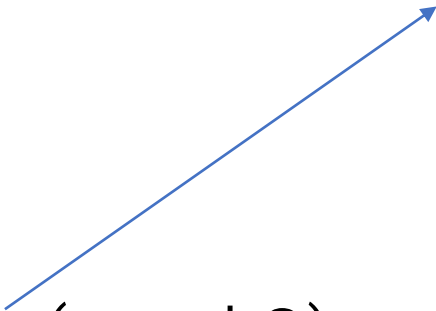
The LPN Generator

What if we take the subset sum generator **but flip some of the output bits?**

$G(A, s, j_1, \dots, j_\lambda)$:

- Compute $y = sA \pmod{2}$
- For all j ,
 - If $j = j_i$ for some i , set $r_j = y_j \oplus 1$
 - Else set $r_j = y_j$
 - Output (A, r)

“Random sparse error”

$$r = sA + e \pmod{2}$$


The LPN Generator

What if we take the subset sum generator **but flip some of the output bits?**

$G(A, s, j_1, \dots, j_\lambda)$:

- Compute $y = sA \pmod{2}$
- For all j ,
 - If $j = j_i$ for some i , set $r_j = y_j \oplus 1$
 - Else set $r_j = y_j$
 - Output (A, r)

Output length: $\lambda \cdot n + n$

Input length: $\lambda \cdot n + \lambda + \lambda \cdot \log(n)$

Set $n = \lambda^2$ and we have stretch!

$$r = sA + e \pmod{2}$$

The LPN Generator

What if we take the subset sum generator **but flip some of the output bits?**

$G(A, s, j_1, \dots, j_\lambda)$:

- Compute $y = sA \pmod{2}$
- For all j ,
 - If $j = j_i$ for some i , set $r_j = y_j \oplus 1$
 - Else set $r_j = y_j$
 - Output (A, r)
- Attack from earlier doesn't work!
 - λ of the equations are wrong, and we don't know which!
- Best known attack: $2^{\lambda / \log(\lambda)}$ time
- Thm (non-trivial): Breaking G is as hard as solving for s !

$$r = sA + e \pmod{2}$$

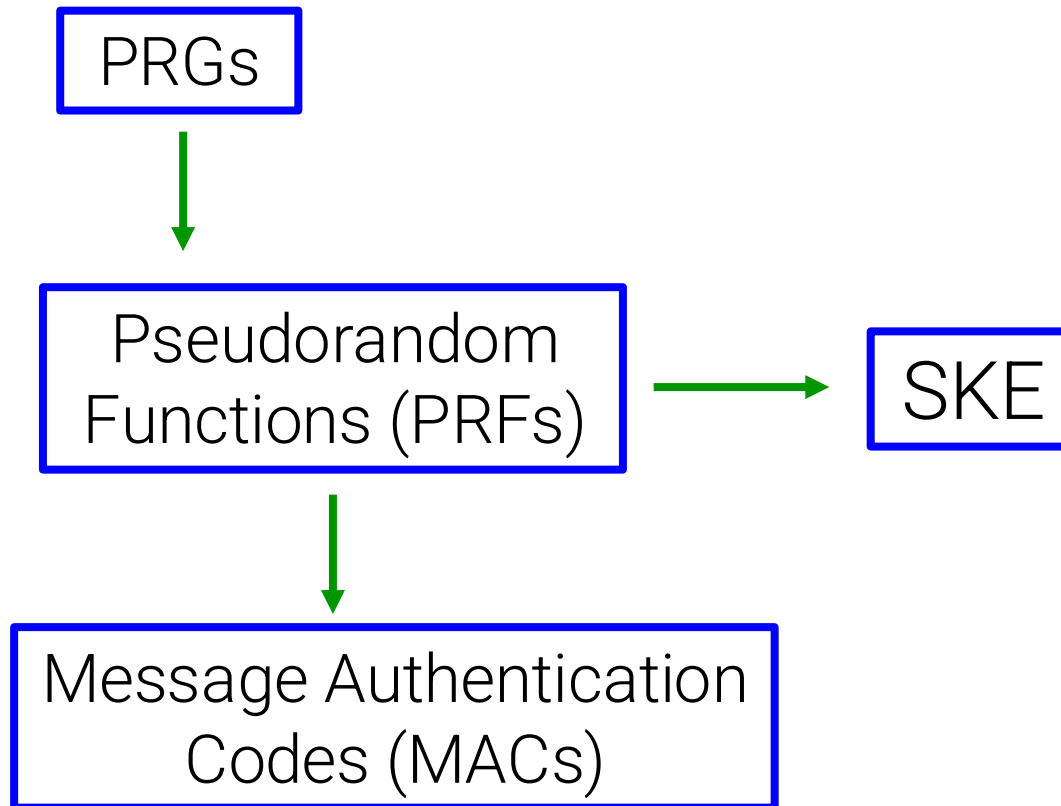
The MQ Generator

What if we take subset sums but add in quadratic terms?

$$G(A, B_1, \dots B_n, s^\top) = (A, B_1, \dots B_n, s^\top A + (s^\top B_1 s, s^\top B_2 s, \dots s^\top B_n s))$$

- General quadratic equations modulo 2 are NP-hard to solve!
- ...but we don't know as much about random quadratic equations
- Unlike LPN, we must restrict $n < \lambda^2$ or else there is an attack! (HW1)

Plan for next week



Secret-Key Encryption (multiple messages)

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m)$$



k

$$m' = \text{Dec}(k, ct)$$

Notation: If (Enc, Dec) is a secret-key encryption scheme with message length $n = n(\lambda)$ and $\mathbf{m} \in (\{0,1\}^n)^t$, we define $\text{Enc}(k, \mathbf{m}) = (\text{Enc}(k, m_1), \dots, \text{Enc}(k, m_t))$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies $t = t(\lambda)$ -message indistinguishability (IND-) security if:

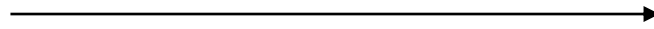
For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m)$$



k

$$m' = \text{Dec}(k, ct)$$

Observation: for $t \geq 2$, encryption can no longer be deterministic!

$$\text{Enc}(\textcolor{red}{k}, m_0), \text{Enc}(\textcolor{red}{k}, m_0) \approx_c \text{Enc}(\textcolor{red}{k}, m_0), \text{Enc}(\textcolor{red}{k}, m_1)$$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies $t = t(\lambda)$ -message indistinguishability (IND-) security if:

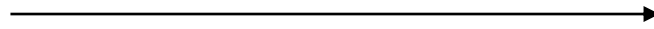
$$\text{For all } \mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right), \quad \boxed{\text{Enc}(\textcolor{red}{k}, \mathbf{m}_0) \approx_c \text{Enc}(\textcolor{red}{k}, \mathbf{m}_1)}.$$

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m; \mathbf{r})$$



k

$$m' = \text{Dec}(k, ct)$$

Notation: If (Enc, Dec) is a secret-key encryption scheme with message length $n = n(\lambda)$ and $\mathbf{m} \in (\{0,1\}^n)^t$, we define $\text{Enc}(k, \mathbf{m}) = (\text{Enc}(k, m_1; \mathbf{r}_1), \dots, \text{Enc}(k, m_t; \mathbf{r}_t))$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies $t = t(\lambda)$ -message indistinguishability (IND-) security if:

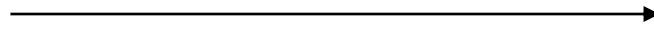
For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m; r)$$



k

$$m' = \text{Dec}(k, ct)$$

Observation: without loss of generality, we can take $n(\lambda) = 1$ (one-bit messages)!

Redefine $\text{Enc}(k, m \in \{0,1\}^n) = (\text{Enc}(k, m_1), \dots, \text{Enc}(k, m_n))$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

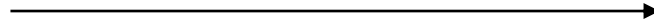
For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in \left(\{0,1\}^{n(\lambda)} \right)^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(k, \mathbf{m}_0) \approx_c \text{Enc}(k, \mathbf{m}_1)$.

Secret-Key Encryption (multiple messages)



k, m

$$ct = \text{Enc}(k, m; r)$$



k

$$m' = \text{Dec}(k, ct)$$

Observation: without loss of generality, we can take $n(\lambda) = 1$ (one-bit messages)!

Exercise: nt -message security (one-bit messages) $\rightarrow$ t -message security (n -bit messages)

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

$$\text{For all } \mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda, \quad \boxed{\text{Enc}(k, \mathbf{m}_0) \approx_c \text{Enc}(k, \mathbf{m}_1)}.$$

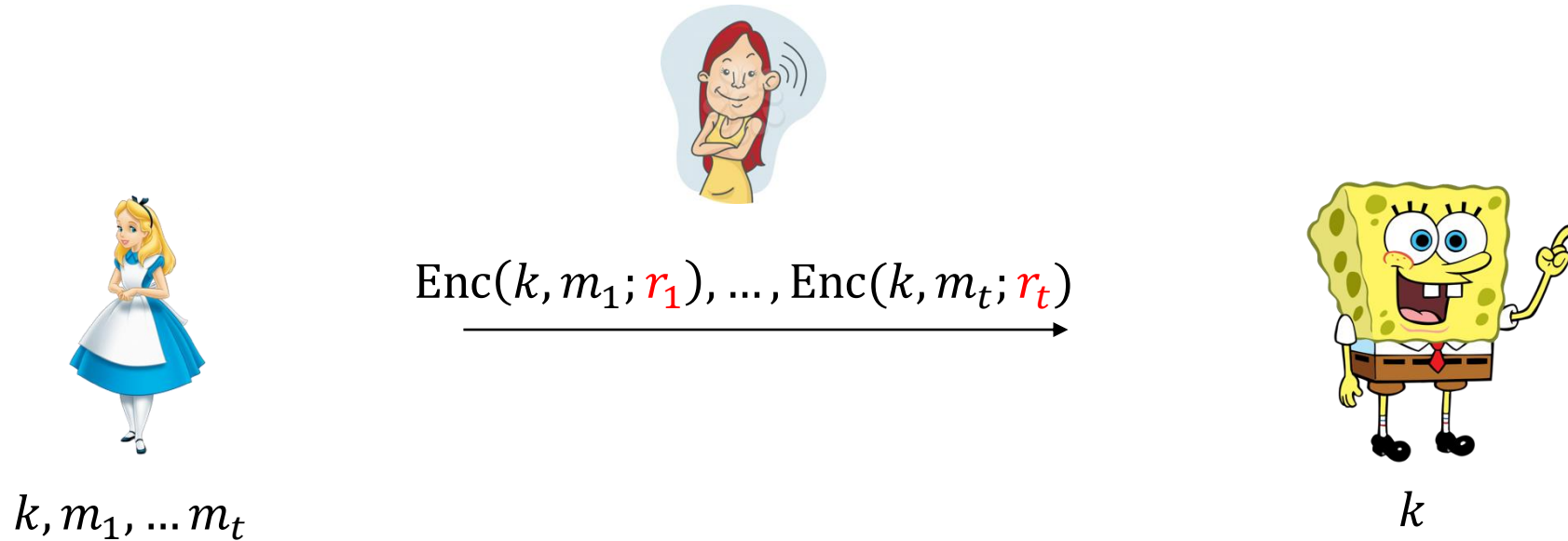
Secret-Key Encryption (multiple messages)

Is this what we want?

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

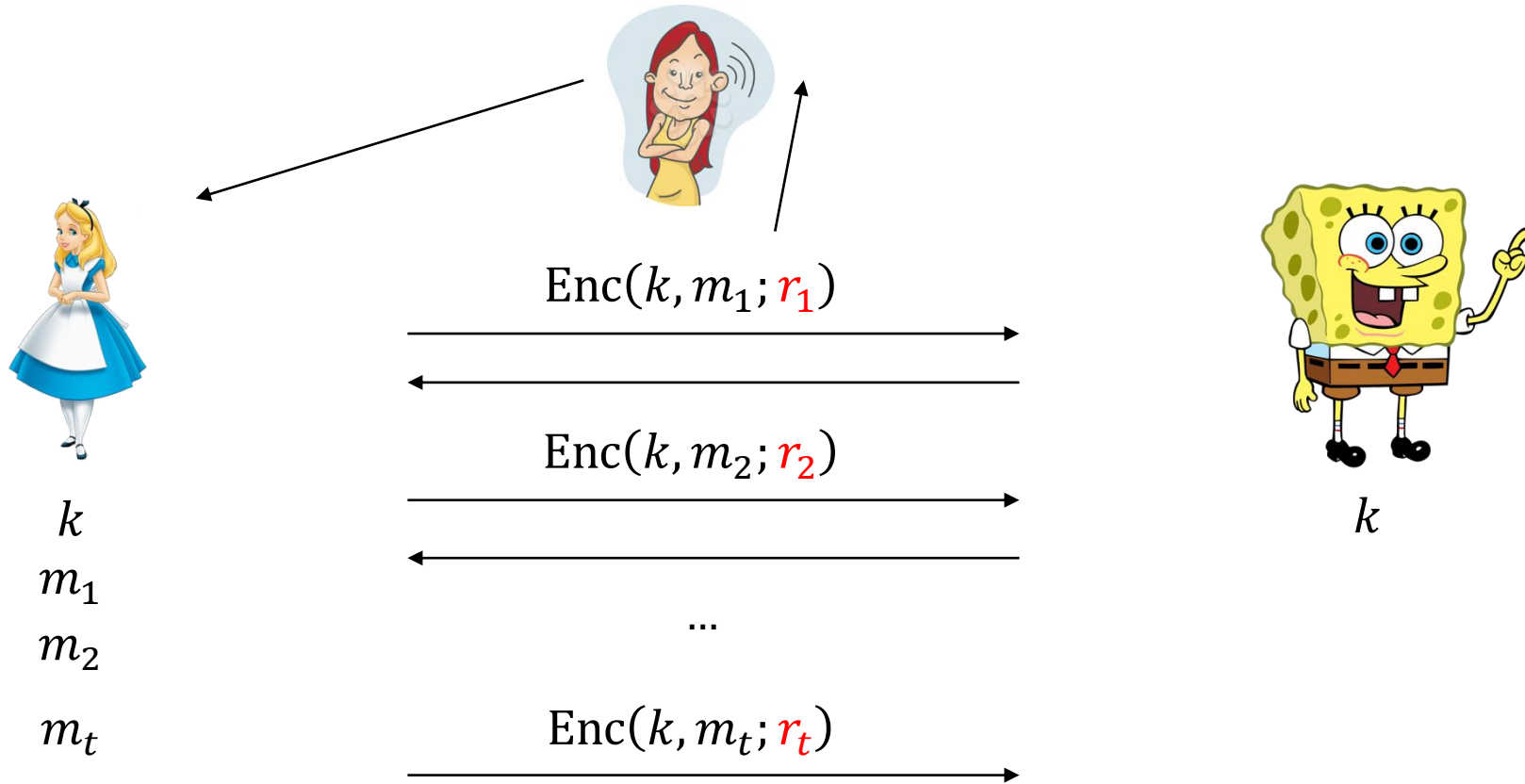
Secret-Key Encryption (multiple messages)



Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies **indistinguishability (IND-) security** if **for all** $t(\lambda) = \text{poly}(\lambda)$,

For all $\mathbf{m}_0 = \left(\mathbf{m}_0^{(\lambda)} \in (\{0,1\}^{n(\lambda)})^{t(\lambda)} \right)_\lambda, \mathbf{m}_1 = \left(\mathbf{m}_1^{(\lambda)} \right)_\lambda$, $\text{Enc}(\mathbf{k}, \mathbf{m}_0) \approx_c \text{Enc}(\mathbf{k}, \mathbf{m}_1)$.

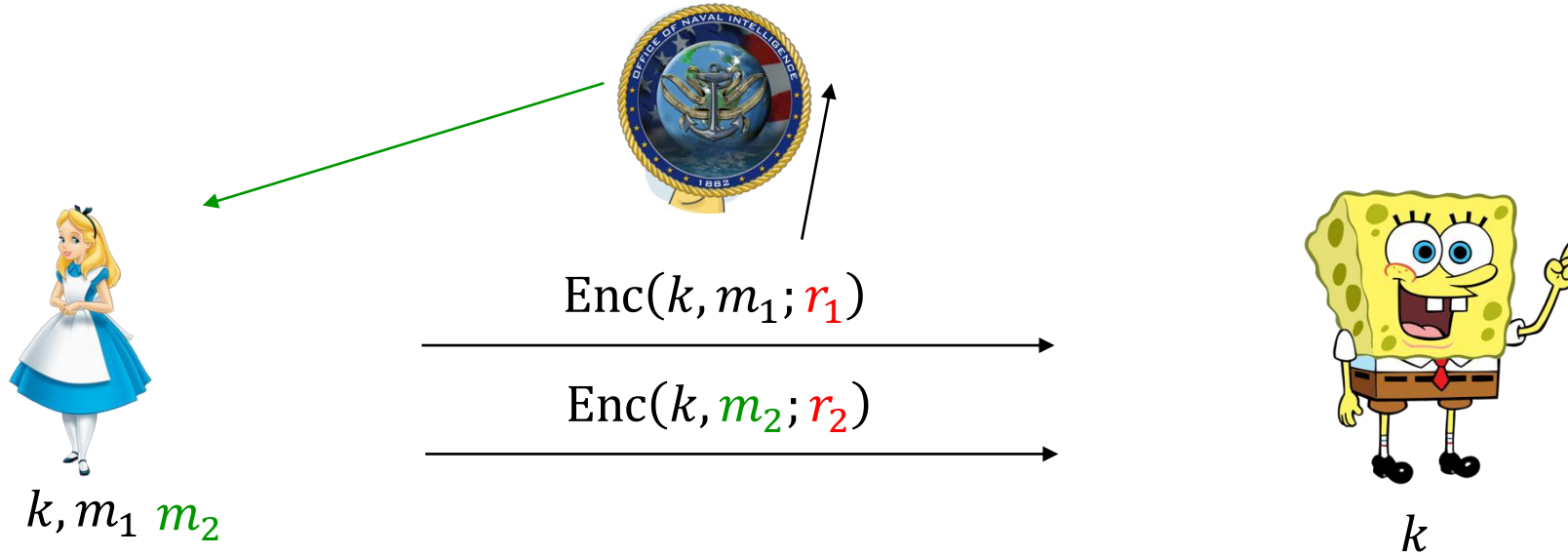
Secret-Key Encryption (multiple messages)



What information **can m_i depend on?**

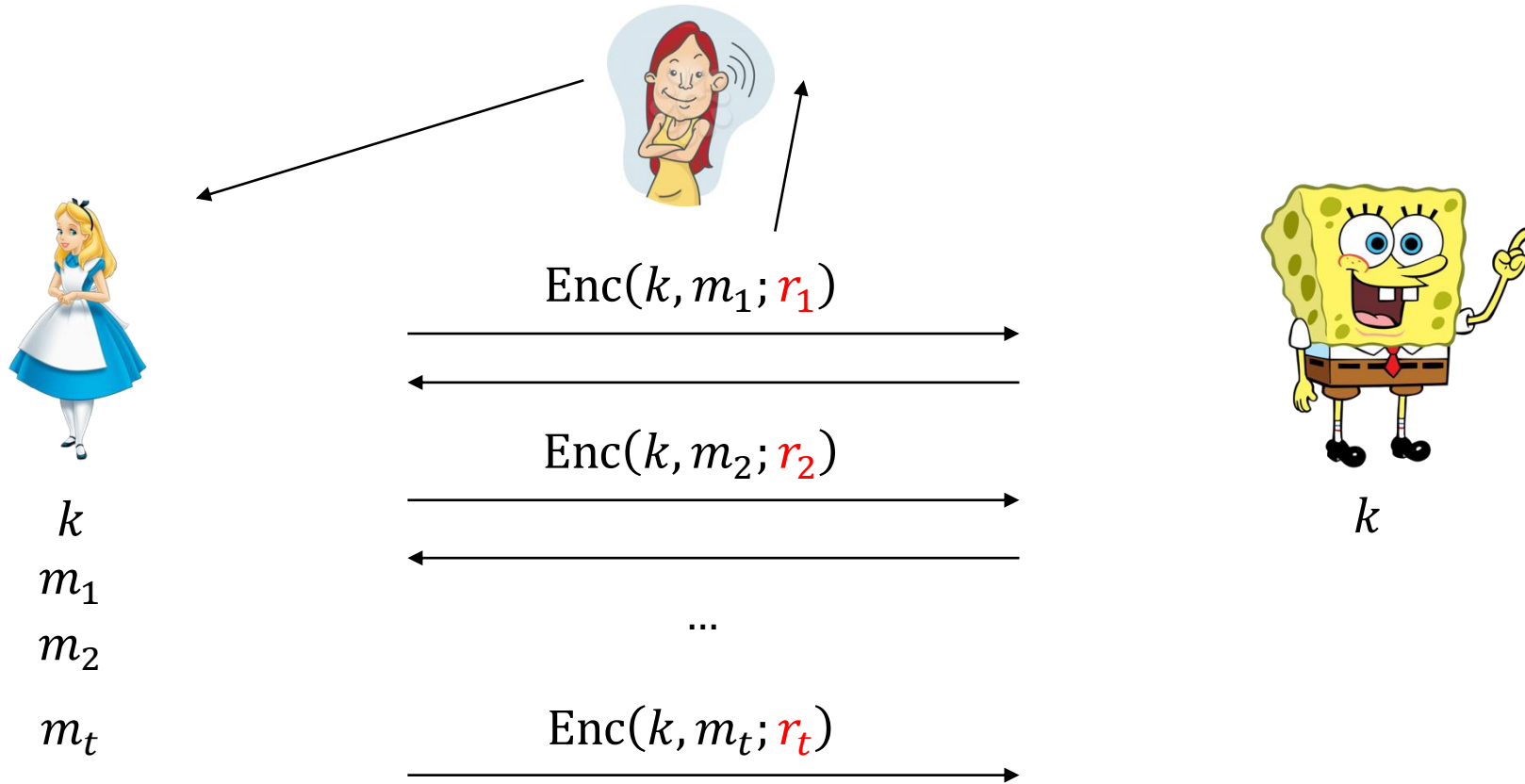
- Messages (encrypted or unencrypted) that Bob sends back to Alice
- The weather on day i
- Actions that the adversary  takes.

An “known plaintext attack”



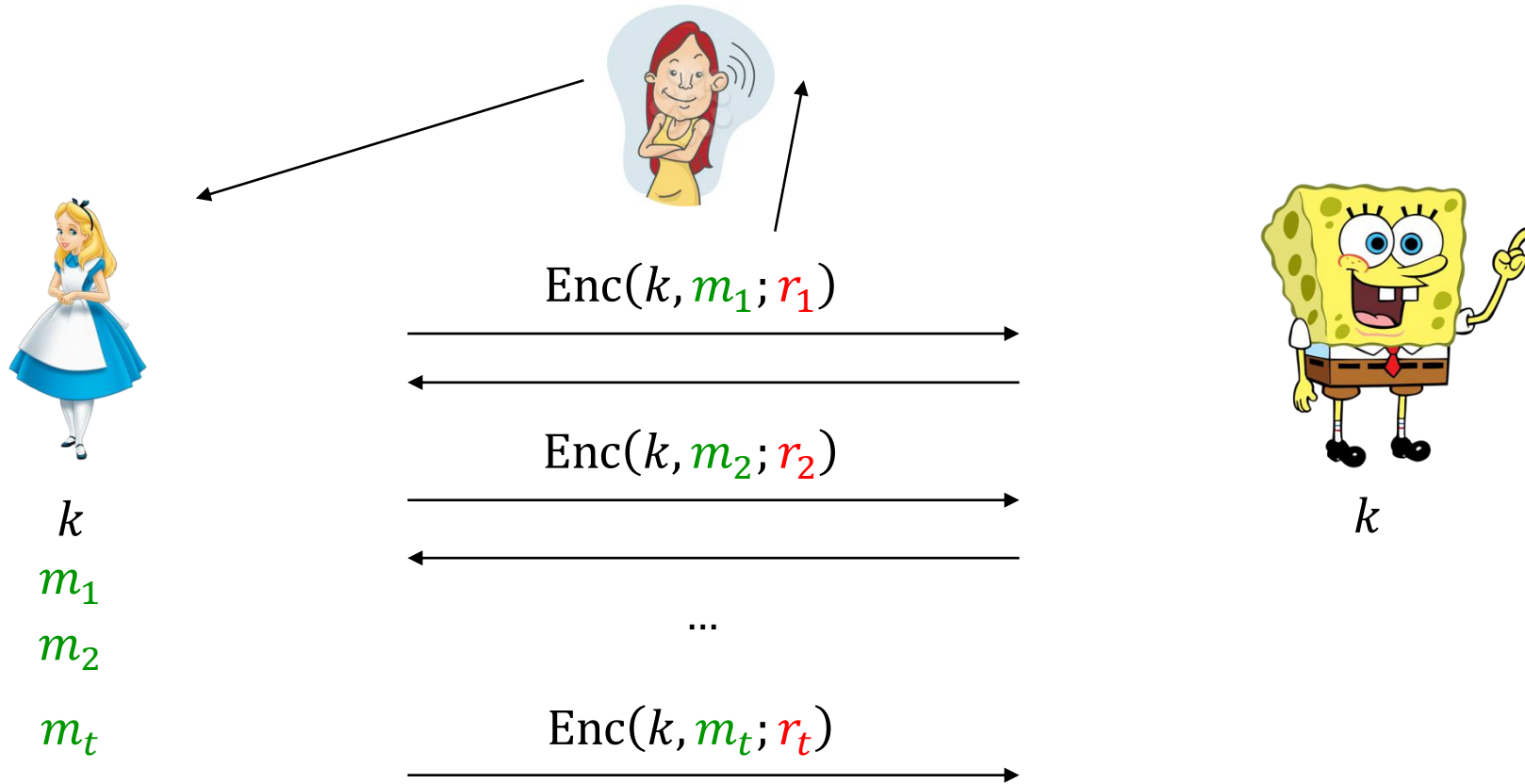
- WWII: US Navy eavesdrops that Japan plans to attack location “AF”
- Previous knowledge: locations near Hawaii have codewords starting with A
- Educated guess: “AF” refers to Midway island
- US releases a fake message about Midway island in the clear (“low supplies”)
- Japan sends reports about “AF” having low supplies.

Secret-Key Encryption (multiple messages)



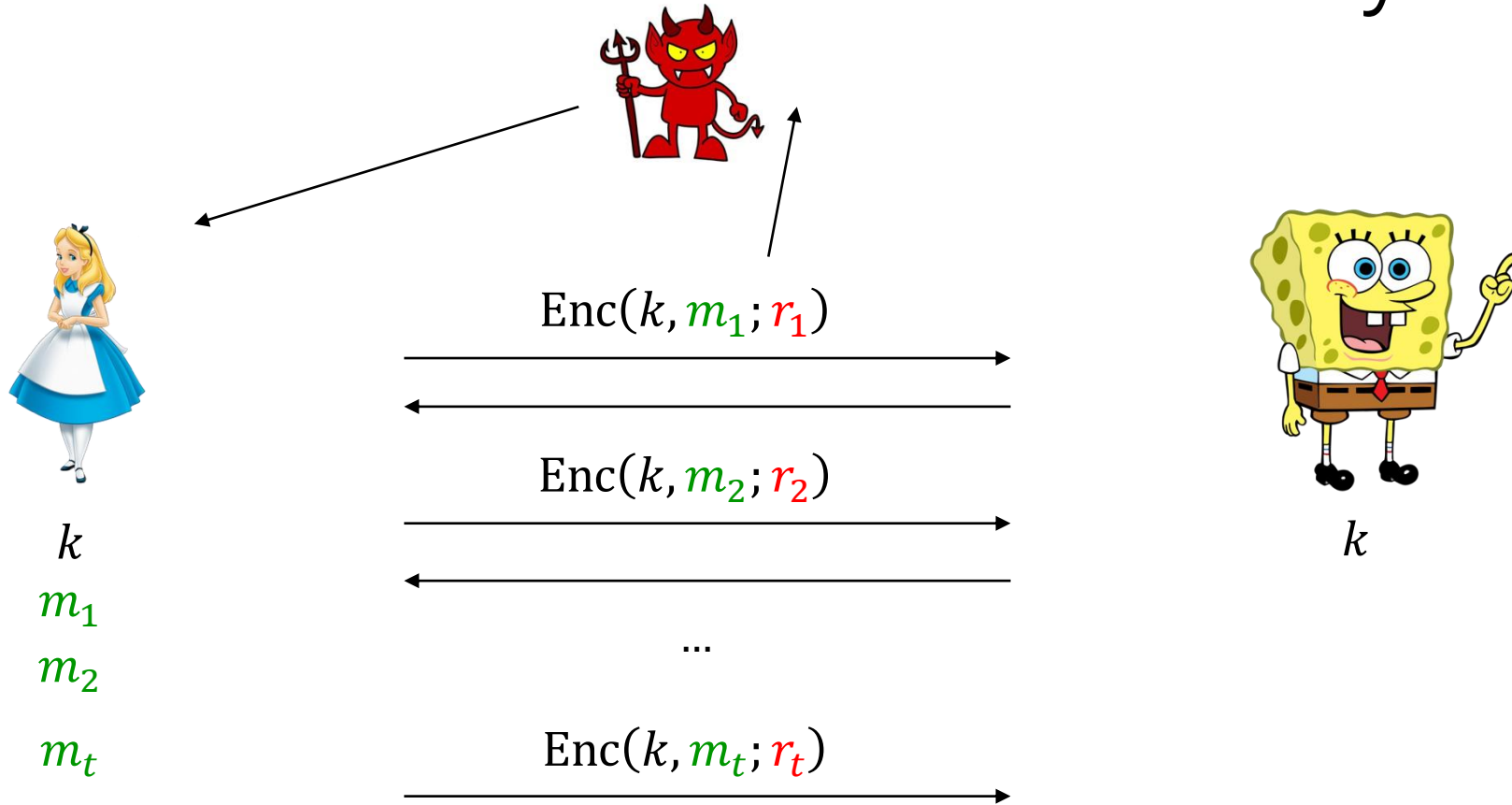
How do we model this dependency?

Chosen-Plaintext Security



Cautious model: adversary has **complete power** over the choice of messages.

Chosen-Plaintext Security



Cautious model: adversary has **complete power** over the choice of messages.

Game-based security definitions